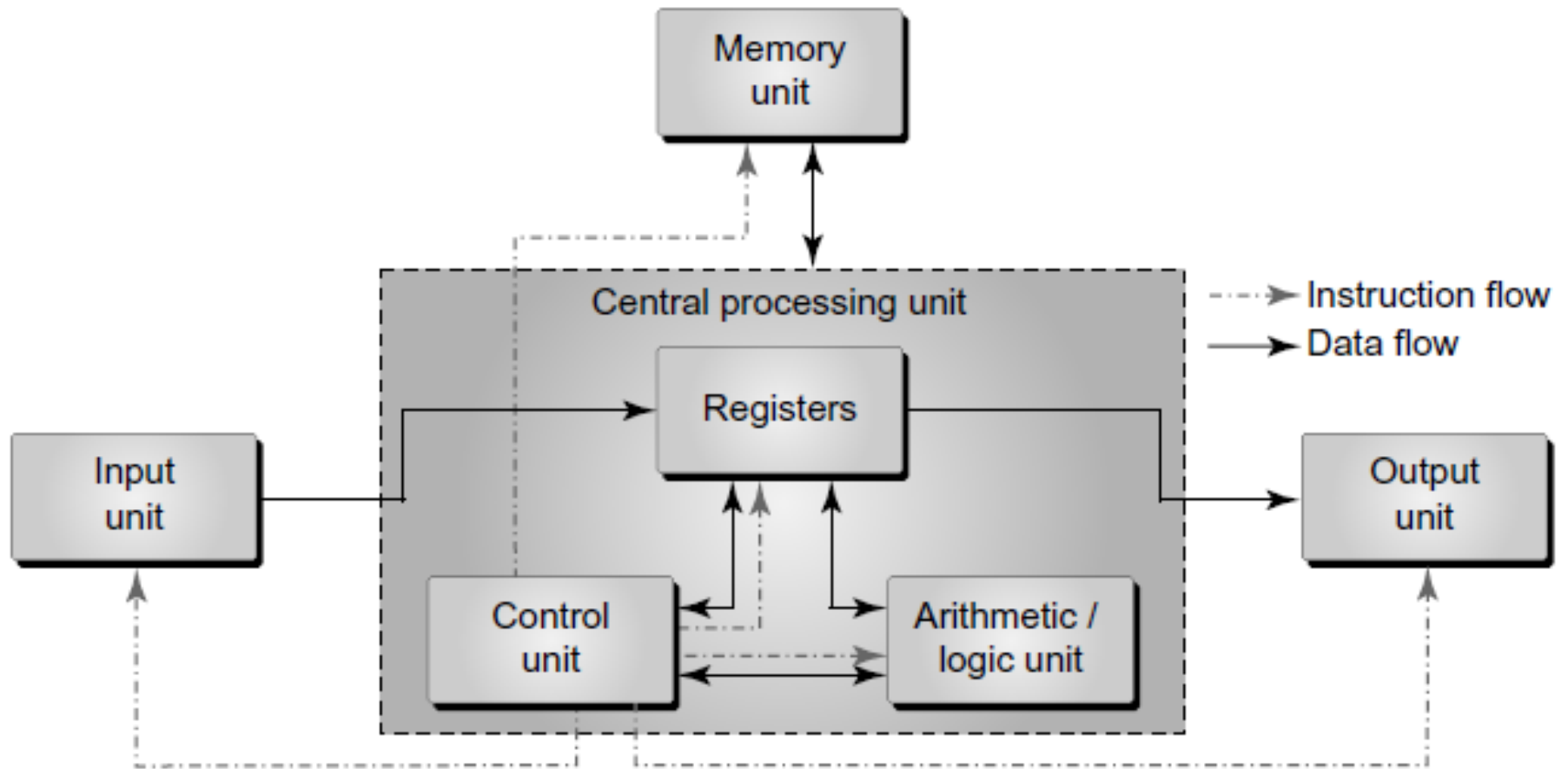


INTRODUCTION TO INFO. & COMM. TECHNOLOGY

21-09-2023

Block Diagram of a computer



Memory

- Part of the computer that holds data and instructions for processing
- It stores program instructions or data for only as long as they relate to the program in operation
- The CPU accesses the main memory in random manner
- → CPU can access any location of this memory to either read information from it or store information in it

Internal Processor Memory

- This memory is placed within the CPU (processor)
- Internal memory usually includes cache memory and special registers
- These can be directly accessed by the processor
- For temporary storage of data and instructions on which the CPU is currently working
- Fastest among all the memories : but most expensive
- Used to compensate for the speed gap between the primary memory and the processor

Primary Memory

- Random Access Memory (RAM)
- Every computer comes with a small amount of ROM that contains the boot firmware (called 'BIOS')
- This holds enough information to enable the computer to check its hardware and load its operating system into RAM at the time of system booting
- RAM → temporarily

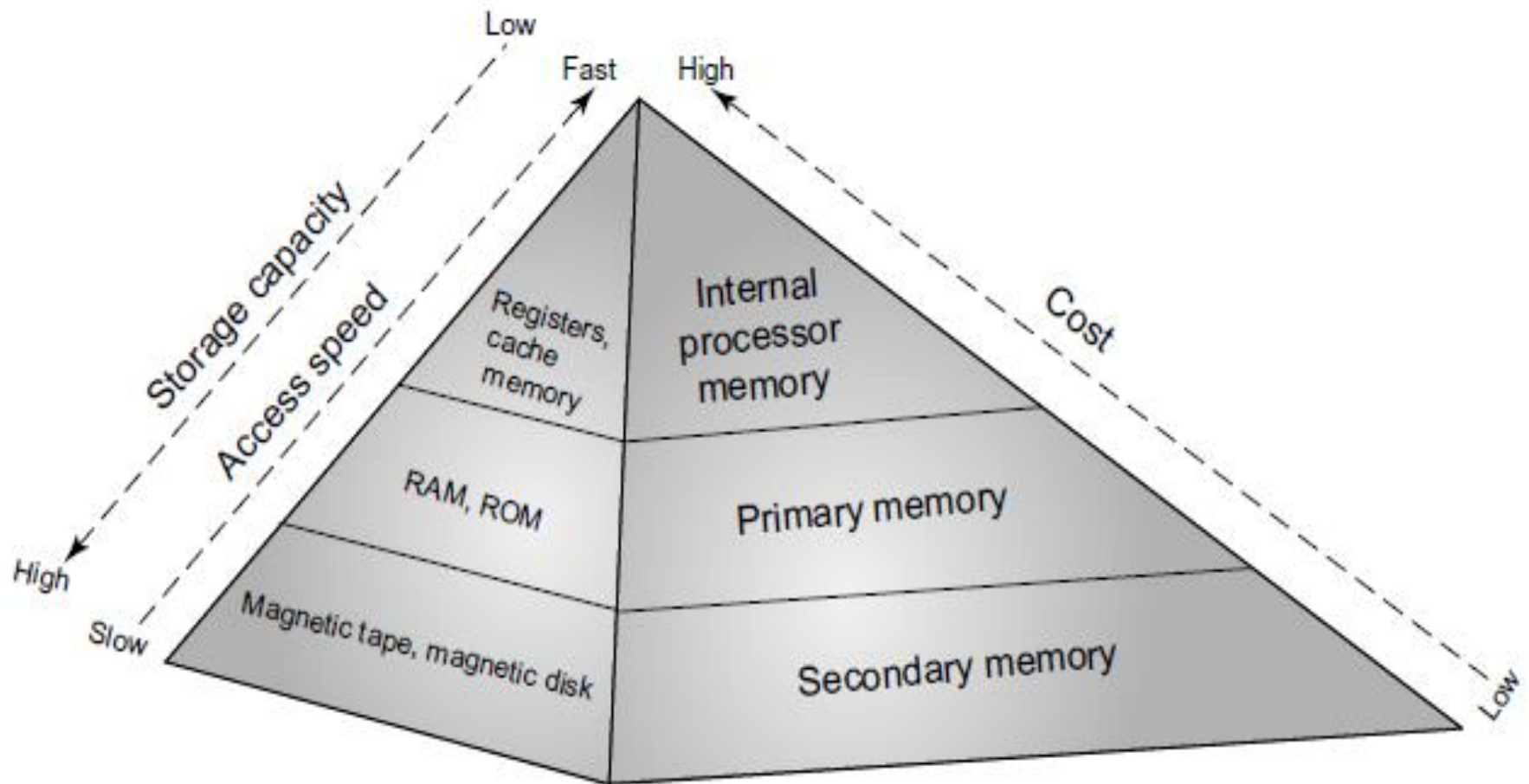
RAM

- To store operating system, application programs and current data
- CPU can reach them quickly and easily
- Volatile → when the power is switched off the data in this memory are lost
- ROM → non-volatile → Even when the computer is switched off, the contents of the ROM remain available

Secondary Memory

- AKA **Auxiliary Memory**
- Backup storage for instructions (computer programs) and data → permanent
- E.g. magnetic disk and magnetic tapes
- Less expensive + larger storage capacity than primary memory
- Secondary memory can also be used as ‘overflow memory’ (also known as ‘virtual memory’), when the capacity of main memory is surpassed
- Secondary memory is not directly accessible to the processor
- **The data and instructions from secondary memory have to be shifted to main memory and then to the processor**

Memory Hierarchy



Memory Hierarchy

- The CPU accesses memory according to a distinct hierarchy
- When the data come from permanent storage (for example, hard disk) → RAM
- With data in primary memory, the CPU can access them more quickly
- The CPU stores the required pieces of data and instructions in processor memory (cache and registers) to process the data at a very high speed

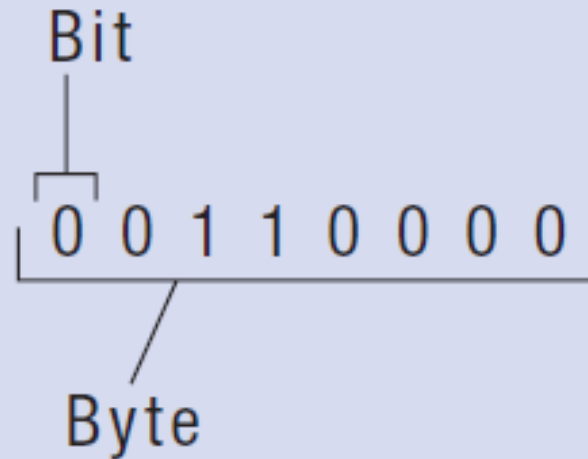
Memory Measurement Units

- **Bit:** The smallest unit of data on a machine and a single bit can hold only one of two values: 0 or 1
- Bit is represented as b
- **Byte:** A unit of eight bits is known as a byte
- It can contain any binary number between 00000000 and 11111111. It is represented as B
- **Kilobyte:** In a decimal system, kilo stands for 1000, but in a binary system, kilo refers to 1024
- Therefore, a kilobyte is equal to 1024 bytes. It is usually represented as KB.

Memory Measurement Units

- **Megabyte:** It comprises 1024 kilobytes, or 1,048,576 bytes. A megabyte can be thought of as a million bytes. Megabyte is the standard unit of measurement for RAM and is represented as MB.
- **Gigabyte:** It consists of 1024 megabytes (1,073,741,824 bytes). It is the standard unit of measurement for hard disks and is often represented as GB.
- **Terabyte:** It refers to 1024 gigabytes and is often represented as TB

Bits & Bytes



Abbreviation	Approximate Size
KB	1 thousand bytes
MB	1 million bytes
GB	1 billion bytes
TB	1 trillion bytes
PB	1,000 terabytes
EB	1,000 petabytes
ZB	1,000 exabytes
YB	1,000 zettabytes

YB = Yottabyte

Language of Computers

- A **bit** is the smallest unit of data that a computer can recognize
- Representing data in a form that can be understood by a digital computer is called *digital data representation*
- → binary can be thought of as the computer's *natural language*

Communicating with computers

- For us to interact with a computer, a translation process from our natural language to 0s and 1s and then back again to our natural language is required
- Data into a computer → it translates the natural-language symbols into binary 0s and 1s
- After processing the data, the computer translates and outputs the resulting information in a form that we can understand
- In this binary system, each 0 or 1 is called a **binary digit** (Bit)

Coding Systems

- While numeric data is represented by the binary numbering system, text-based data is represented by binary coding systems
 - ASCII, EBCDIC, and Unicode → coding schemes
- .To represent all characters —such as numbers, letters, and special characters

ASCII & EBCDIC

- American Standard Code for Information Interchange:
- Coding system traditionally used with personal computers
- *EBCDIC (Extended Binary-Coded Decimal Interchange Code)* was developed by IBM, primarily for use with mainframes
- There is 8-bit *extended versions* of ASCII
- The extended ASCII and EBCDIC represent each character as a unique combination of 8 bits (1 byte), which allows 256 unique combinations

Unicode

- **Unicode** is a universal international coding standard designed to represent text-based data written in any ancient or modern language
- To enable us to work with alphabets in Chinese, Greek, Hebrew, Russian etc.
- It is a longer code, consisting of 8 to 32 bits per character
- It can represent over one million characters, which is more than enough unique combinations to represent the standard characters in all the world's written languages
- Also allows us to work with thousands of mathematical and technical symbols, punctuation marks, and other symbols and signs

Unicode

- **Unicode** is used by most Web browsers for Web pages and Web applications (Google data, for instance, is stored exclusively in Unicode)
- Latest versions of Microsoft Windows, Microsoft Office, etc. also use Unicode, as do modern programming languages, such as Java and Python
- Unicode is updated regularly to add new characters and new languages not originally encoded
- The most recent version is *Unicode 8.0*

Number Systems

- Decimal
- Binary
- Octal
- Hexadecimal

Decimal Number System

- The number system we all use in everyday life
- we refer to this system as a **base 10 system**
- The number 23, for example, is 2 tens and 3 ones ($2*10 + 3*1$)
- The number 4,657 is 4 thousands, 6 hundreds, 5 tens, and 7 ones ($4*1000 + 6*100 + 5*10 + 7*1$)
- In the number system, the 'radix' (also called 'base') tells the number of symbols used in the system

Types of number systems

Number System	Radix Value	Set of Digits
Decimal	$r = 10$	(0, 1, 2, 3, 4, 5, 6, 7, 8, 9)
Binary	$r = 2$	(0, 1)
Octal	$r = 8$	(0, 1, 2, 3, 4, 5, 6, 7)
Hexadecimal	$r = 16$	(0, 1, 2, 3, 4, 5, 6, 7, 8, 9, A, B, C, D, E, F)

Decimal Number System

- 0 - 9 → 10 digits
- Base = 10
- The **base** is the number we are acting on, and the **exponent** (the number written, usually, as a superscript) tells the reader what to do with the base
- 3^5 → we call 3 the base and 5 the exponent

Base of a number

- Any number raised to a power can be written as X^a
- X is the base and a is the exponent
- In computer programming, a base with an exponent is represented as X^a
- where X is the base, the caret (^) indicates raising a number to an exponent, and a is the exponent

Expanded Notation

- Any number in the decimal system can be written as a sum of each digit multiplied by the value of its column
- This is called **expanded notation**

Expanded Notation

The number 23 in the decimal system actually means the following:

$$\begin{array}{rclcl} & 3 * 10^0 & = & 3 * 1 & = & 3 \\ + & 2 * 10^1 & = & 2 * 10 & = & 20 \\ \hline & & & & & 23 \end{array}$$

Therefore, 23 can be expressed as $2 * 10^1 + 3 * 10^0$.

Binary No. System (Contd.)

- Binary comes from Latin word **bi** which means two
- In binary, there are two digits → 0 and 1
- A single **one** or **zero** is called a bit
- Bit ← contraction from the words **binary** and **digit**
- 4 bits = nibble
- 8 bits = byte
- A word is usually 2 bytes or sixteen bits
- With new processors word can be 32 or even 64 bits (quad word)
- **Base = 2**

Decimal to Binary - Class Practice

- $(12)_{10}$
- $(202)_{10}$
- $(02)_{10}$
- $(123)_{10}$
- $(1010)_{10}$

Class Practice - Decimal to Binary

- Convert following Decimal numbers to Binary:

1. 1002

2. 28

3. 131

4. 12

5. 100

Binary to Decimal

i. 1001 1001

ii. 1100 1111

iii. 0011 0011

iv. 0010 0001

v. 1101 1111

Ref Chapters

- Book 2
- Chp 5